

AI Software Code Reviewer & Secure Development Agent

Technology & Architecture — why each choice, what it solves

Qwen2.5-Coder / Ollama

Semgrep

RAG

Redis

Verifier

OpenTelemetry

Team Alpha • Hackathon PS #06

Local-first • no API key required • runs offline

1. What it does

One automated pipeline for secure code review

The problem

- Manual review is slow and misses security bugs.
- Static tools flag issues but don't explain or fix them.
- Naive AI fixes hallucinate unsafe 'best practices'.

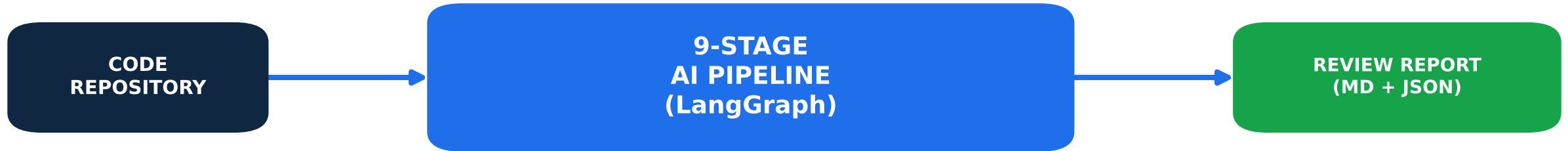
Our solution — a multi-agent pipeline that:



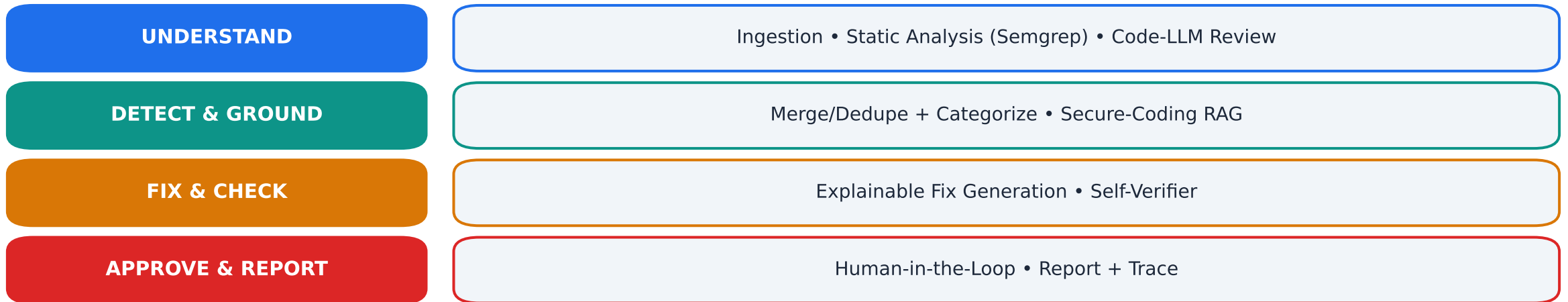
Input: a code repository → **Output: explainable review report + verified fixes**

2. Architecture — the big picture

9 specialist agents on a LangGraph pipeline over one shared state



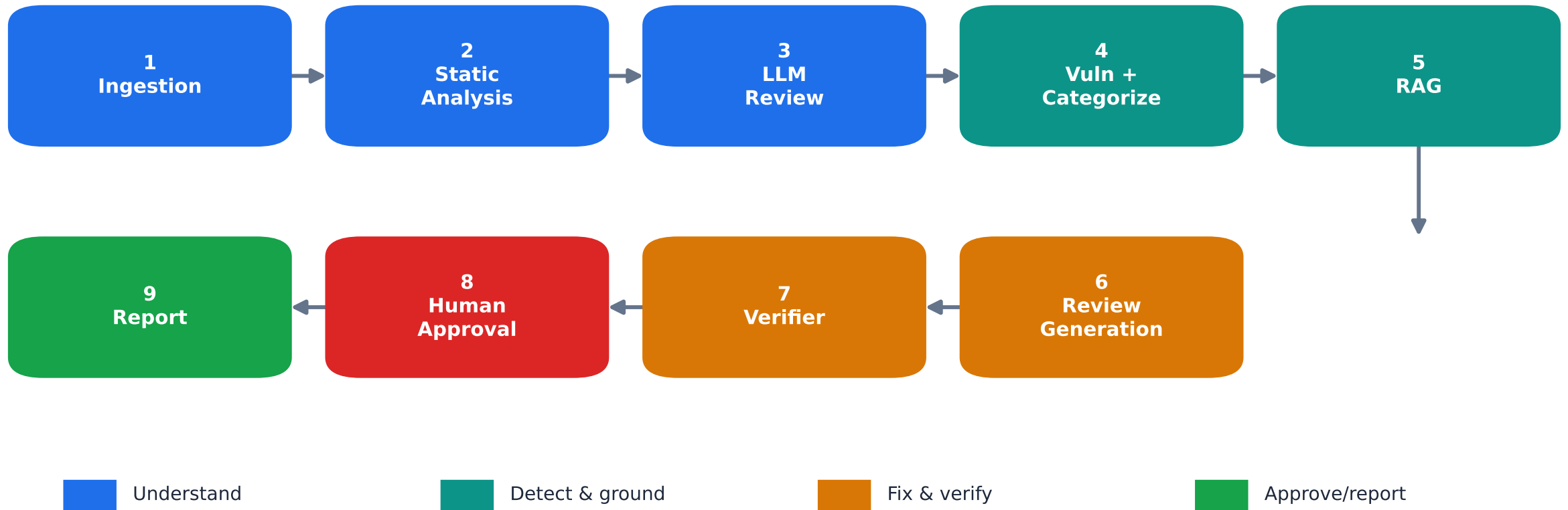
Every stage reads and adds to one typed ReviewState — flows left to right



Cross-cutting: Redis cache • OpenTelemetry/LangSmith tracing • optional LoRA (offline)

3. The pipeline flow

Data moves through 9 stages, in order



Linear & traceable: each stage consumes the previous stage's output.

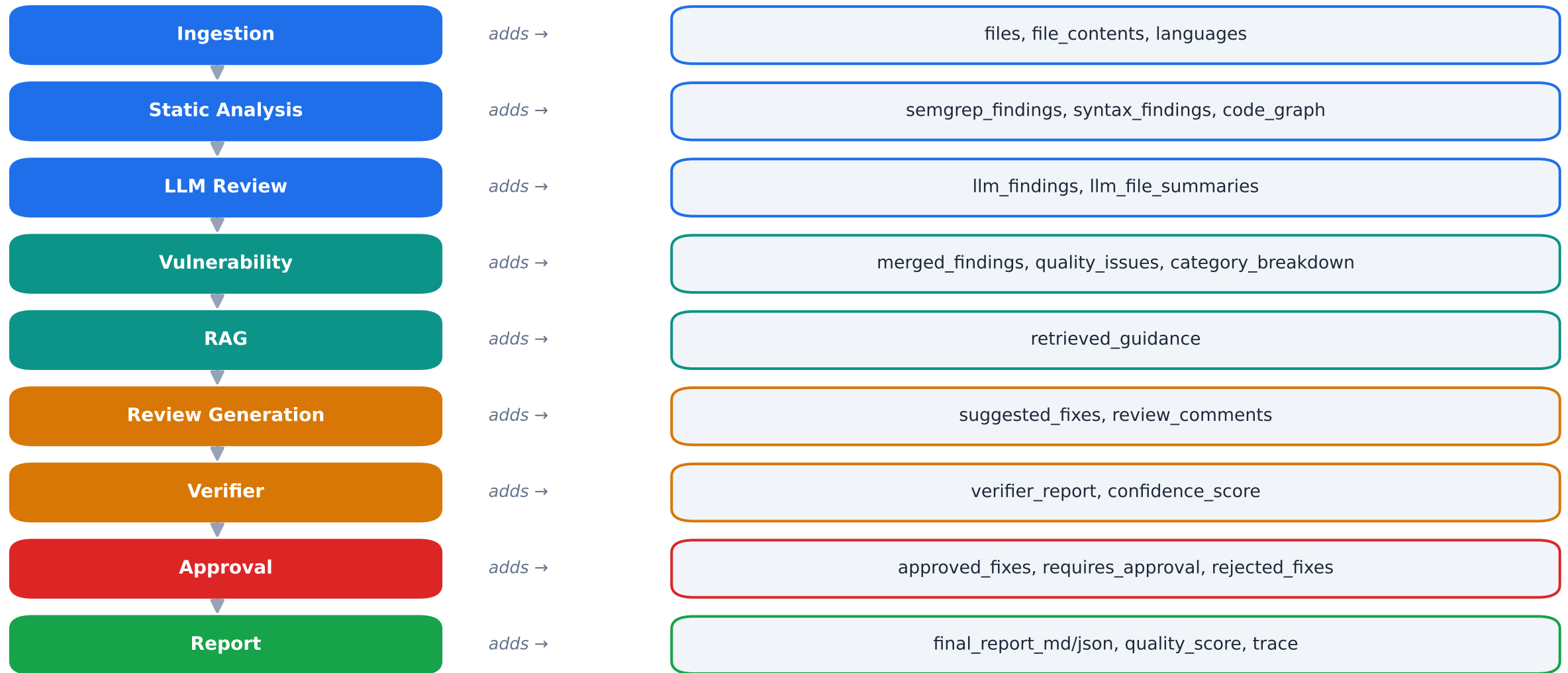
4. The 9 agents

Each LangGraph node is one specialist (agents/*.py)

1	Ingestion	Discover files, read contents, detect language
2	Static Analysis	Semgrep rulesets + code graph + syntax errors
3	LLM Review	Qwen reviews each file for logic/quality bugs
4	Vulnerability	Merge+dedupe; categorize syntax/logic/security/quality
5	RAG	Retrieve grounded OWASP/CWE + dataset guidance
6	Review Generation	Explainable comment + minimal grounded fix
7	Verifier	Syntax + no-op + Semgrep regression re-scan
8	Approval	Route CRITICAL/ERROR or flagged fixes to a human
9	Report	Assemble artifacts + agent trace (MD + JSON)

5. Data flow — the shared ReviewState

What each stage adds to one state object



6. Technology choices at a glance

Every major tech maps to a purpose

LangGraph	Orchestrate 9 agents over one typed state
Qwen2.5-Coder + Ollama	Local code-LLM: semantic review + fixes
Semgrep	Fast deterministic static vuln detection
AST code graph	Structural context (GNN stand-in)
RAG (FAISS/TF-IDF)	Ground fixes in OWASP/CWE + datasets
Redis	Cache LLM calls: repeat runs instant
Verifier	Validate AI fixes before applying
OpenTelemetry / LangSmith	Per-agent observability / tracing
LoRA (offline)	Optional fine-tune toward secure coding
Streamlit	Interactive dashboard + toggles

The next slides explain each choice precisely.

7. LangGraph

Multi-agent orchestration

PURPOSE

Wire the 9 agents into an explicit StateGraph that passes one typed ReviewState from node to node.

WHY WE CHOSE IT

Explicit graph + shared state contract makes the pipeline deterministic, traceable, and easy to extend; integrates natively with LangSmith.

WHAT IT SOLVES

Coordinating specialized agents (the PS 'Agentic Challenge') without brittle ad-hoc function chaining.

ALTERNATIVE WE PASSED ON

Hand-rolled function calls: no state schema, no tracing hooks, hard to extend.

8. Qwen2.5-Coder + Ollama

The Code-LLM (semantic review + fixes)

PURPOSE

A code-specialised LLM that reads each file for logic bugs and writes minimal fixes, served locally by Ollama.

WHY WE CHOSE IT

Qwen2.5-Coder is code-trained (named in the PS) and beats a general model at code; Ollama runs it locally — no API key, no data egress, free, with native constrained-JSON (format:json) for reliable structured output.

WHAT IT SOLVES

Semantic bugs static analysis can't see; producing grounded, parseable fixes on a small local model.

ALTERNATIVE WE PASSED ON

Hosted API (cost, key, data leaves the machine) — kept pluggable for those who want it.

9. Semgrep

Static & semantic analysis

PURPOSE

Scan the repo with OWASP/CWE rulesets for known vulnerability patterns (SQLi, secrets, eval, weak crypto, ...).

WHY WE CHOSE IT

Fast, deterministic, high-precision, industry-standard rules; ships an offline fallback ruleset so it works with no internet.

WHAT IT SOLVES

High-confidence detection of known vuln classes — the deterministic half that complements the LLM's semantic half.

ALTERNATIVE WE PASSED ON

CodeQL (heavier, slower setup); pure-LLM detection (hallucination-prone).

10. AST Code Graph

Structural context (honest GNN stand-in)

PURPOSE

Build a function/class/call graph + flagged dangerous sinks and feed it to the LLM as structural context.

WHY WE CHOSE IT

AST/regex is cheap, deterministic and offline; it is an explicit stand-in for the PS's 'Graph Neural Network', which is NOT implemented.

WHAT IT SOLVES

Gives the LLM code structure instead of raw text, improving review quality.

ALTERNATIVE WE PASSED ON

A trained GNN (needs labeled graph data + GPU) — documented swap-in, out of scope.

11. RAG (Retrieval-Augmented Generation)

Grounded, explainable fixes

PURPOSE

For each finding, retrieve authoritative secure-coding guidance (OWASP/CWE + dataset samples) and inject it into the fix prompt.

WHY WE CHOSE IT

Grounding kills hallucinated 'best practices'; gives every fix a citable source; updatable by editing a JSON file (no retraining); lets a small local model punch above its weight.

WHAT IT SOLVES

The worst failure mode for a security tool: a confident but wrong fix.

ALTERNATIVE WE PASSED ON

Rely on the model's memory alone (hallucinates, can't cite, hard to update).

12. Why RAG is optional

A user toggle, not a fixed stage

RAG can be switched off from Streamlit, the CLI (--no-rag), or USE_RAG.

Compare grounded vs. ungrounded

Run the same repo with RAG on and off to show the value of grounding — a clean ablation/demo.

Speed & simplicity

Skip retrieval when you just want the model's raw behaviour or a faster pass.

Graceful fallback

With RAG off, fix generation falls back to general secure-coding best practice (logged as a notice).

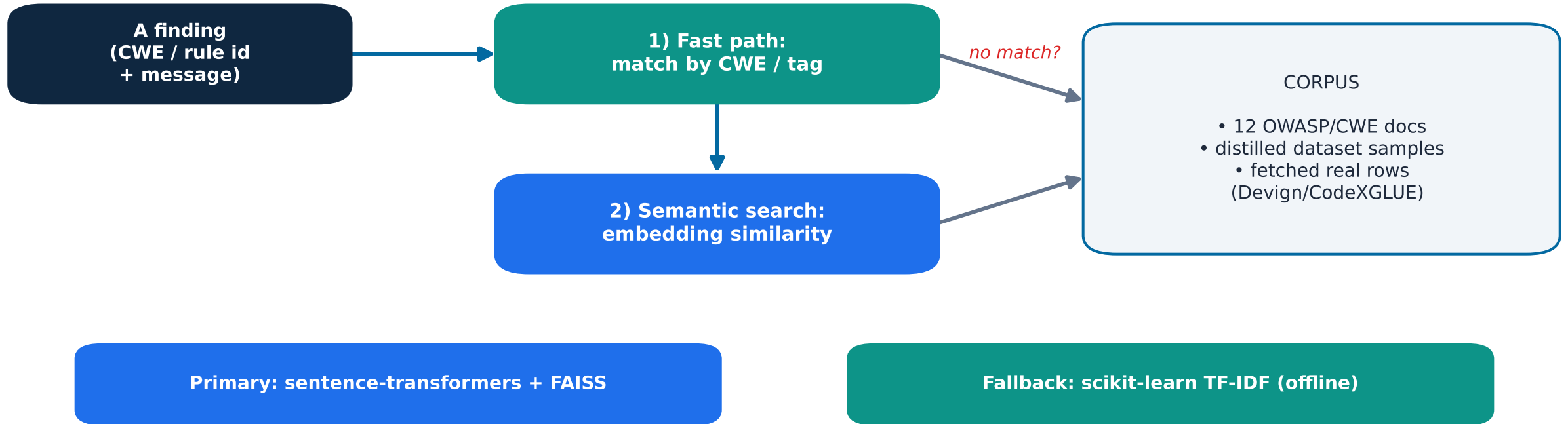
Flexibility for evaluation

Reviewers/mentors can inspect exactly how much RAG contributes to fix quality.

Default is ON (grounded). Off is a deliberate, surfaced choice — not a limitation.

13. Inside the RAG engine

How grounded guidance is retrieved



Auto-falls back to TF-IDF if embedding models can't download — RAG works offline.

Retrieved snippets are injected into the fix prompt; the LLM must ground its fix in them.

14. Redis

LLM-response caching layer

PURPOSE

Cache every LLM completion keyed by (provider, model, prompt, mode); a repeat request is served from cache instead of re-querying the model.

WHY WE CHOSE IT

LLM calls are the slow, expensive step; identical calls recur across runs. Redis is fast with TTL; redislite gives a zero-setup embedded fallback; it can also back a vector store later.

WHAT IT SOLVES

Latency & cost of repeated identical LLM calls — measured 4.7s → 0.001s on a cache hit.

ALTERNATIVE WE PASSED ON

No cache (every run re-pays full LLM latency) — painful on a local model.

15. Verifier / Self-Reflection

Validate AI fixes before applying

PURPOSE

Independently check each generated fix: syntax parse, no-op detection, and a Semgrep regression re-scan confirming the rule no longer fires.

WHY WE CHOSE IT

LLMs hallucinate and produce inconsistent code; an independent check is essential before a fix is ever called 'applicable'.

WHAT IT SOLVES

Hallucinated / lazy / broken fixes — yields a per-fix + overall confidence score.

ALTERNATIVE WE PASSED ON

Trusting raw LLM output (unsafe for security remediation).

16. Human-in-the-Loop Approval

The safety gate

PURPOSE

Route CRITICAL/ERROR-severity or verifier-flagged fixes to a human; low-risk verified fixes can auto-approve.

WHY WE CHOSE IT

Security changes need human accountability; nothing high-risk should apply automatically.

WHAT IT SOLVES

Unsafe auto-application of high-impact changes — keeps a human in control.

ALTERNATIVE WE PASSED ON

Fully autonomous apply (dangerous and non-auditable for security).

17. Finding Categorization

Distinct, surfaced categories

PURPOSE

Tag every finding as syntax / logic / security / quality and surface each in its own report section, with a category_breakdown.

WHY WE CHOSE IT

Reviewers triage by type; syntax errors were previously only parser metadata, now promoted to first-class CRITICAL findings.

WHAT IT SOLVES

The review requirement for clear separation of bug/vuln/quality classes.

ALTERNATIVE WE PASSED ON

One undifferentiated list of findings (hard to triage).

18. Observability (OpenTelemetry / LangSmith)

Per-agent tracing

PURPOSE

Wrap every node so each run records a span (name, duration, status, output) into state['trace']; export to OTLP (Phoenix/Jaeger) or LangSmith.

WHY WE CHOSE IT

You can't improve what you can't see; the PS names Phoenix/MLflow. Set TRACING_BACKEND to export real spans; default is in-process only.

WHAT IT SOLVES

No visibility into per-agent timing/status/failures.

ALTERNATIVE WE PASSED ON

Only printing final output + error strings (no timing, no external trace).

19. LoRA Fine-Tuning

Optional offline training track

PURPOSE

Parameter-efficient fine-tuning of a Code-LLM toward secure-coding recommendations; trains tiny adapter weights, saved to disk.

WHY WE CHOSE IT

LoRA trains ~1-2% of params (small, cheap adapter) vs. full fine-tune; it is the PS's 'trained model' technique, offered ALONGSIDE RAG, not replacing it.

WHAT IT SOLVES

Specialising a model's default behaviour without a full retrain.

ALTERNATIVE WE PASSED ON

Full fine-tune (GPU-heavy, static, can't cite sources) — and it never runs at app time.

20. Datasets — complete information

The named vulnerability datasets and how each is used

Dataset	Purpose	How used here
Devign	C/C++ vuln functions	RAG (fetched via CodeXGLUE)
Big-Vul (MSR'20)	CVE-labeled vulns	RAG (distilled sample)
CodeXGLUE	Code intelligence	RAG source (Devign defect-detection)
Juliet (NIST)	CWE weakness cases	RAG (distilled sample)
DiverseVul	Large-scale vulns	RAG (distilled sample)
ManySStuBs4J	Java bug fixes	RAG (distilled sample)
SAP Project-KB	Java bug benchmark	RAG (distilled sample)
OWASP Benchmark	Tool benchmark	Reference / evaluation

Representative, not exhaustive: RAG uses ~10 distilled samples + an optional capped subset streamed from ONE dataset (default Devign). Full multi-GB datasets are NOT ingested in full, and none is loaded live at review time.

21. How to input datasets

Three ways to feed the RAG corpus

1 Bundled samples

Curated CWE-labeled rows already in the repo.

```
rag/datasets/vuln_samples.jsonl (auto-loaded)
```

2 Fetch real rows

Stream vulnerable functions from HuggingFace and cache them.

```
python rag/fetch_datasets.py --dataset <hf-id> --limit 200
```

3 Drop-in your own

Add any *.jsonl of {dataset,cwe,vulnerable,guidance,tags};
the loader globs the folder automatically.

```
rag/datasets/my_data.jsonl
```

Toggle the whole feature with `USE_DATASET_RAG`. Fetched cache is auto-included, no code change.

Note: the CODE to review is a different input — pass it with `--repo <path>` (CLI) or the Streamlit path box.

22. Stack & how to run

Local-first, no API key

Core stack

Orchestration	LangGraph
Static analysis	Semgrep
Cache	Redis / redislite
UI	Streamlit

Code-LLM	Ollama - Qwen2.5-Coder
RAG	FAISS / TF-IDF + datasets
Tracing	OpenTelemetry / LangSmith
Fine-tune	PEFT LoRA (offline)

Run

```
pip install -r requirements.txt
ollama serve && ollama pull qwen2.5-coder:1.5b

python main.py --repo ./sample_repo          # CLI review
python main.py --repo ./sample_repo --no-rag # ungrounded baseline
streamlit run app.py                          # dashboard + toggles
python rag/fetch_datasets.py                  # real dataset rows into RAG
```

23. Scope & honest limitations

What is real vs. a stand-in

No trained GNN / vuln model

Structure is AST/regex; detection is Semgrep + LLM.

Dataset RAG is representative

Distilled samples + a capped fetched subset, not full ingestion.

Tracing is span-level

Real OTLP/LangSmith export, but not token-level LLM tracing.

LoRA is offline & optional

Needs a GPU; the app never trains at runtime.

Local-repo review only

No GitHub/GitLab/Docker/FastAPI integration.

Small local model ceiling

qwen2.5-coder:1.5b can truncate; larger models improve quality.

Stated plainly on purpose — credibility comes from precise claims.

24. Why this design

The through-line

Hybrid detection

Deterministic Semgrep + semantic LLM catch different bugs.

Grounded & explainable

RAG-cited fixes, not hallucinated best practice.

Self-checking + safe

Verifier re-validates; humans gate high-risk changes.

Observable

Every agent traced; exportable to Phoenix/LangSmith.

Local-first & offline

No API key; Ollama + Semgrep/RAG fallbacks.

Honest & extensible

Clear stand-ins; datasets, LoRA, tracing all pluggable.

Finds → Grounds → Verifies → Guards → Explains